Lab 4:

VibeCoding

Name: Sagar K **SECTION: G**

SRN: PES2UG24CS424

Github Link: https://github.com/sagark-05/CS424_Traffic_Escape.git

Videos (Before and After) and Updated Code are available in the above GitHub repository.

Chat history:

Lab 4 VibeCoding: Traffic Escape

Conversation Record

- Project: Traffic Escape, in `50_traffic-escape/`
- Assistant: GitHub Copilot
- Date: 2026-09-30
- Purpose: Record the feature requests, implementation decisions, and checks discussed for this assignment.
- Chat share URL: Not available from this workspace. Add the GitHub Copilot Chat share URL here if your submission requires one.

The user requests below are reproduced in order. Assistant entries capture the response and implementation outcome for each request; routine tool protocol is omitted.

1. Investigate and Fix Crash Behavior

User

> Analyze the existing Traffic Escape game code and identify the cause of the broken behavior where the game immediately ends when the player crashes into a car.

> Fix only this broken behavior without adding any new features from the README. Preserve the existing game architecture, controls, movement, traffic, scoring, and winning logic.

>

> Explain what you changed and why, then provide the updated code for the required file(s).

Assistant response and outcome

The player spawned with its center on the divider between two lanes. Because the car hitboxes extend toward the lane edges, cars in adjacent lanes could overlap the player and cause a false collision. The spawn was moved to the center of a lane. The collision response itself, including game-over on a real overlap, was left intact. A geometry check confirmed that adjacent-lane cars no longer overlap the player while same-lane cars still do.

Updated file: [game_engine.py](50_traffic-escape/game/game_engine.py).

2. Add Three Lives

User

> Implement Task 1 from the README: add a 3-life system to the Traffic Escape game.

>

> Requirements:

>

> - The player starts with 3 lives.

> - When the player gets hit by a car, lose 1 life.

> - The game should not end when the player loses a life if lives remain.

> - Reset the player's position after a crash.

> - When lives reach 0, show the game-over state.

> - Display the remaining lives on the game screen.

> - Keep the existing controls, traffic, scoring, and winning logic.

> - Do not implement any other README tasks yet.

> Explain the changes and provide the updated code for the required file(s).

Assistant response and outcome

Three lives were initialized on restart and shown in the HUD. A car hit now removes one life, respawns the player, and removes the car that caused the hit so it cannot immediately deduct another life. Game-over is set at zero lives. Existing movement, controls, scoring, and winning logic were preserved. Headless checks passed for losing lives, respawning, game-over at zero, and restoring three lives on reset.

Updated file: [game_engine.py](50_traffic-escape/game/game_engine.py).

3. Add the Moving-Raft Safe Lane

User

> Implement Task 2 from the README: add a safe lane where the player must hop on a moving log/raft to cross.

>

> Requirements:

>

> - Add a safe lane containing moving logs/rafts.

> - The logs/rafts should move horizontally across the lane.

> - The player must be able to stand on a log/raft and move with it.

> - If the player is in the safe lane but not on a log/raft, they should not be able to safely cross it.

> - Handle falling off the log/raft appropriately.

> - Integrate the new lane with the existing game and collision system.

> - Keep the existing 3-life system and other functionality working.

> - Do not implement the high-score or day/night features yet.

> Explain the changes and provide the updated code for the required file(s).

Assistant response and outcome

A `Raft` entity was added with horizontal movement and edge bouncing. The engine draws a central water lane, removes traffic entering that lane, carries a supported player with the raft, and applies the existing life-loss and respawn behavior when the player enters the water unsupported or falls off. Checks covered raft movement, riding, falling, traffic handling, and car-hit lives.

The user then reported:

> hey iam not able to move to the top

The assistant traced the issue to water detection using any overlap of the player's tall rectangle. That counted the player as entering the water before they reached raft height. Water entry was changed to use the player's center, raft support uses horizontal overlap, and the center raft was oriented toward the starting position. A straight-up simulation then reached the top with all three lives.

Updated files: [game_engine.py](50_traffic-escape/game/game_engine.py) and [traffic.py](50_traffic-escape/game/traffic.py).

4. Add Persistent High Scores

User

> Implement Task 3 from the README: add a persistent high-score table.

>

> Requirements:

>

> - Track the player's score.

> - Save high scores to a JSON file.

> - Keep the top 5 scores only.

> - Load the saved scores when the game starts.

> - Update the high-score table when a game ends or the player wins.

> - Display the top 5 scores clearly in the game.

> - The scores must persist across different game sessions.

> - Handle the case where the JSON file does not exist yet.

> - Keep all existing features working.

> - Do not implement the day/night cycle yet.

> Explain the changes and provide the updated code for the required file(s).

Assistant response and outcome

The engine now loads and saves the five highest displayed scores (`score // 10`) to `50_traffic-escape/high_scores.json`. A missing or malformed JSON file starts with an empty table. A score is recorded once when the player wins or reaches game-over. The top five are shown in the HUD and end-game overlay. Temporary-file checks passed for empty-file handling, ordering, limiting to five, persistence, and both terminal outcomes.

Updated file: [game_engine.py](50_traffic-escape/game/game_engine.py).

5. Add the Day/Night Cycle

User

> Implement Task 4 from the README: add a day/night cycle.

>

> Requirements:

>

> - The game should automatically switch between day and night every 30 seconds.

> - Change the background/environment appearance to clearly show day and night.

> - During night mode, cars should have visible headlights.

> - The headlights should be visible further ahead of the cars.

> - The day/night cycle should continue automatically while playing.

> - Keep the existing 3-life system, moving log/raft lane, high-score system, controls, scoring, and winning logic working.

> - Do not remove any existing features.

> Explain the changes and provide the updated code for the required file(s).

Assistant response and outcome

The engine toggles day/night from elapsed time every 30 seconds, updates the environment colors, and labels the current mode in the HUD. Cars draw directional translucent headlight beams extending 180 pixels ahead, with bright lamps visible at the front. Checks passed for 30- and 60-second transitions, normal update-loop timing, and beams on cars traveling in both directions.

Updated files: [game_engine.py](50_traffic-escape/game/game_engine.py) and [traffic.py](50_traffic-escape/game/traffic.py).

6. Prepare the Assignment Chat Record

User

> I need to submit my Lab 4 VibeCoding assignment.

>

> Prepare a complete, submission-ready chat history document based on our conversation for this project.

>

> I need all this chat history to be included in copilot_context.md

Assistant response and outcome

This document records the project requests, implementation outcomes, and verification results in chronological order. A share URL cannot be generated from the workspace; add the actual GitHub Copilot Chat share URL above if the assignment requires it.

Verification Notes

- Collision geometry: adjacent-lane overlap was eliminated; genuine same-lane overlap remains a collision.
- Lives: hit, respawn, zero-lives game-over, and restart behavior passed headless checks.
- Rafts: horizontal movement, carrying, unsupported-water falls, car handling, and the straight-up crossing passed checks.
- High scores: missing-file behavior, persistence, descending order, top-five limit, win save, and game-over save passed temporary-file checks.
- Day/night: 30-second transitions, continued update-loop operation, and directional headlights were checked with mocked time and in-memory surfaces.
- The editor reported that `pygame` could not be resolved by its selected analysis environment; the runtime checks imported Pygame successfully.